

「README」

Crypto Diary / 投資日記

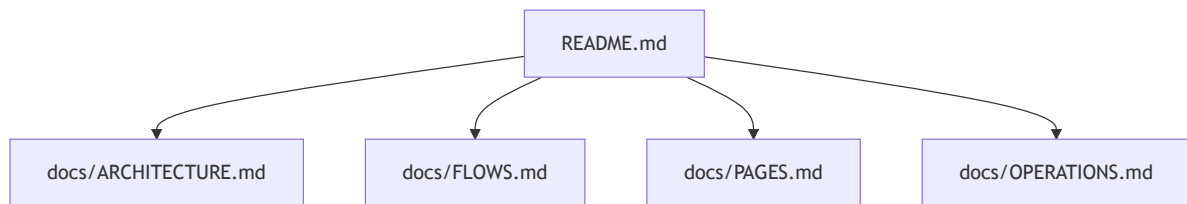
Streamlit + Google Sheets 的唯讀投資日記儀表板。狀態：已部署於 Streamlit Cloud；網頁端是 read-only / 唯讀模式。

Crypto Diary 集中展示加密貨幣交易紀錄、ETF 投資紀錄、其他投資紀錄，以及每日自動投資排程產生的投資計劃與歷史日記。所有新增與修改都透過 Google Sheet、外部排程或匯入腳本完成，Streamlit app 只負責讀取與呈現。

Live App

- Streamlit Cloud: <https://crypto-diary-byprx9psuxdvbuymohhgr.streamlit.app/>
- GitHub repo: <https://github.com/wsx5031060310guy/crypto-diary>

專案文件



- [系統架構](#)：專案總覽、技術棧、元件關係、目錄與資料模型。
- [操作與業務流程](#)：Streamlit 連線、總覽彙總、日記閱讀、交易檢視與匯入流程。
- [頁面 / 路由 / 模組清單](#)：單頁 Streamlit 結構、tabs、外部 API 呼叫與主要函式。
- [安裝、執行、部署、維護](#)：本機啟動、Secrets、環境變數、Streamlit Cloud 與常見維護操作。

快速開始

```
git clone https://github.com/wsx5031060310guy/crypto-diary.git
cd crypto-diary
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

本機需建立 `.streamlit/secrets.toml`，或在 Streamlit Cloud App Settings → Secrets 設定 Google Service Account 與 Sheet 資訊。

功能

- 總覽 Dashboard
 - 交易筆數
 - 每日排程日記篇數
 - 累計買入 / 賣出金額與淨投入
 - 各分類、各資產持倉摘要
- 每日排程日記
 - 顯示 `daily-investment-journal` cron 的歷史紀錄
 - 可閱讀完整 Markdown 投資計劃
 - 支援 Crypto / ETF / 其他分類篩選
- 交易紀錄
 - 讀取 Google Sheet `trades` worksheet
 - 顯示 `asset`, `side`, `amount`, `price`, `total`, `note`
- 唯讀資料流
 - Streamlit app 使用 `spreadsheets.readonly` scope
 - App 內沒有新增交易或寫入 Google Sheet 的 UI
 - 寫入統一由 Google Sheet 本身、排程或匯入腳本完成

架構

每日投資排程 / 手動 Google Sheet 編輯

```
├── Google Sheet: trades
│   └── 交易紀錄 canonical store
├── Google Sheet: journal_entries
│   └── 每日排程投資計劃 / 歷史日記
└── Streamlit App
    └── 唯讀讀取 + dashboard / journal viewer
```

Google Sheet Schema

trades

欄位	說明
timestamp	交易時間，例如 2026-05-08 09:00:00
asset	資產代號，例如 BTC，ETH，SPY，QQQ
side	buy 或 sell
amount	數量
price	單價（TWD）
total	總額（TWD）
note	策略、訊號或備註

journal_entries

欄位	說明
date	日記日期
run_time	排程執行時間
category	Crypto，ETF，其他，可用逗號分隔
source	來源，例如 Hermes cron daily-investment-journal
title	日記標題
summary	摘要
content_markdown	完整 Markdown 內容
github_path	對應 GitHub journal path，例如 daily/YYYY-MM-DD.md
cron_output_path	本機 cron output path，如有
tags	標籤，例如 daily-schedule, investment-plan, crypto, etf

Streamlit Secrets

在 Streamlit Cloud 或本機 `.streamlit/secrets.toml` 設定：

```
[gcp_service_account]
type = "service_account"
project_id = "..."
private_key_id = "..."
private_key = "-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
client_email = "...@...iam.gserviceaccount.com"
client_id = "..."
auth_uri = "https://accounts.google.com/o/oauth2/auth"
token_uri = "https://oauth2.googleapis.com/token"
auth_provider_x509_cert_url = "https://www.googleapis.com/oauth2/v1/certs"
client_x509_cert_url = "..."
universe_domain = "googleapis.com"

[sheet]
id = "Google Sheet ID"
worksheet = "trades"
journal_worksheet = "journal_entries"
```

不要把 service account JSON、`.streamlit/secrets.toml`、private key 或 token commit 到 GitHub。

匯入每日排程歷史紀錄

`scripts/import_investment_journals.py` 會把既有投資日記匯入 Google Sheet `journal_entries`，並自動去重。

預設匯入來源：

- ~/ai-investment-journal/daily/*.md
- ~/ai-investment-journal/shadow/*.md
- ~/.hermes/cron/output/751ec9850858/*.md

執行：

```
python scripts/import_investment_journals.py
```

在 Mike 的 Hermes 環境中可使用既有 crypto bot venv：

```
/Users/mike-hermes-ai/.hermes/crypto_bot/venv/bin/python \  
/Users/mike-hermes-ai/projects/crypto-diary/scripts/import_investment_journals.py
```

可用環境變數覆蓋預設路徑與設定：

- CRYPTO_BOT_SHEET_ID
- CRYPTO_BOT_GCP_CRED
- CRYPTO_BOT_JOURNAL_WORKSHEET
- AI_INVESTMENT_JOURNAL_DIR
- HERMES_INVESTMENT_CRON_OUTPUT

每日排程整合

Hermes cron `daily-investment-journal` 每天 09:00 會：

1. 研究 World Monitor / Finance Monitor、BTC、ETH、SPY、QQQ 等市場資訊。
2. 產生 Crypto + ETF 投資策略。
3. 更新 `ai-investment-journal` repo 的 `daily/YYYY-MM-DD.md` 與 portfolio 狀態。
4. 同步日記內容到 Google Sheet `journal_entries`。
5. 本 Streamlit app 讀取 Google Sheet 並展示結果。

安全原則

- Web app 只讀取 Google Sheet，不提供寫入按鈕。
- Google API 憑證只放在 Streamlit Secrets 或本機安全路徑。
- Repo 不保存 service account JSON、Telegram token 或其他 secret。
- 若需要手動新增資料，請直接改 Google Sheet；不要在 app 內新增寫入能力。

「ARCHITECTURE.md」

系統架構

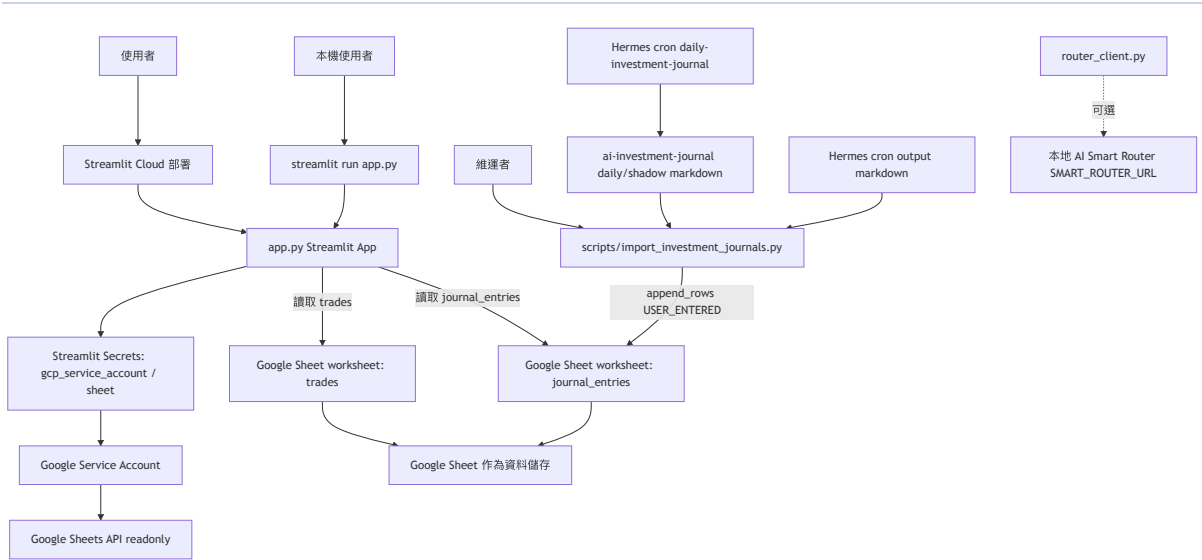
專案總覽

Crypto Diary / 投資日記是一個 Streamlit + Google Sheets 的唯讀投資日記儀表板。它把加密貨幣、ETF、其他投資交易，以及每日排程產生的投資決策日記集中展示，適合需要查閱投資紀錄、每日策略輸出與分類概覽的個人投資者或維護者。網頁端不提供寫入功能；交易與日記資料由 Google Sheet、外部排程或匯入腳本寫入。

技術棧

類別	實際使用	來源	用途
UI 框架	Streamlit	requirements.txt, app.py	建立單頁互動式儀表板、sidebar、tabs、metrics、charts
語言	Python	.py 原始碼	應用與維運腳本
資料處理	pandas	requirements.txt, app.py	整理交易、日期、分類、彙總與圖表資料
Google Sheets client	gsread	requirements.txt, app.py, scripts/import_investment_journals.py	讀取與寫入 Google Sheet worksheet
Google Auth	google-auth	requirements.txt, app.py, scripts/import_investment_journals.py	使用 Service Account 授權 Google Sheets / Drive
資料儲存	Google Sheets	README.md, app.py	trades 與 journal_entries worksheet 作為 canonical store
部署	Streamlit Cloud	README.md	對外部署 Streamlit app
外部排程	Hermes cron daily-investment-journal	README.md, 匯入腳本欄位	產生每日投資日記並同步到 Sheet
輔助 HTTP client	httpx	router_client.py	呼叫本地 AI Smart Router；目前主 app.py 未引用，且 requirements.txt 未列出

架構圖



主要目錄結構

路徑	用途
app.py	Streamlit 主程式。讀取 Google Sheets，渲染投資日記 dashboard、日記閱讀器、交易紀錄與寫入說明。
router_client.py	本地 AI Smart Router 的薄 client，提供 <code>_pick_model()</code> 與 <code>chat()</code> ；目前沒有被 <code>app.py</code> 匯入。
scripts/import_investment_journals.py	維護匯入腳本。掃描本機 markdown 日記與 Hermes cron output，去重後寫入 <code>journal_entries</code> 。
requirements.txt	Python runtime 依賴。
README.md	專案入口說明、Live App、schema、開發與安全資訊。
docs/	本次建立的技术文件。

資料模型概覽

資料模型位於 Google Sheet worksheet，repo 沒有 Prisma schema、SQL migration 或 ORM model。app.py 定義 TRADE_HEADER 與 JOURNAL_HEADER；匯入腳本也以相同 journal_entries header 寫入。

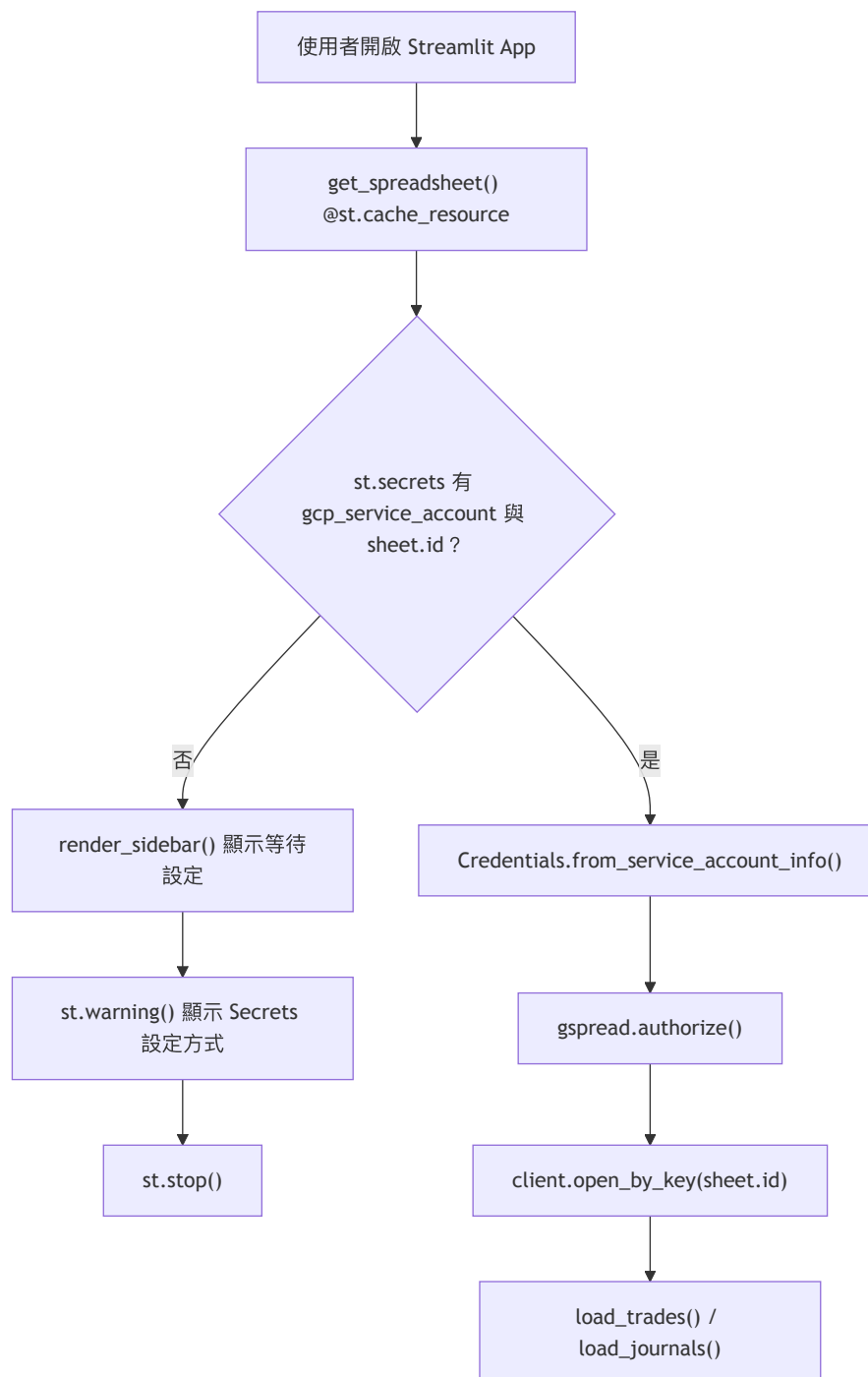
TRADES	
string	timestamp
string	asset
string	side
float	amount
float	price
float	total
string	note

JOURNAL_ENTRIES	
string	date
string	run_time
string	category
string	source
string	title
string	summary
string	content_markdown
string	github_path
string	cron_output_path
string	tags

兩個 worksheet 目前沒有外鍵或程式層關聯。app.py 只用 category 篩選與彙總；trades.asset 會透過 classify_asset() 對應到 Crypto、ETF 或其他。

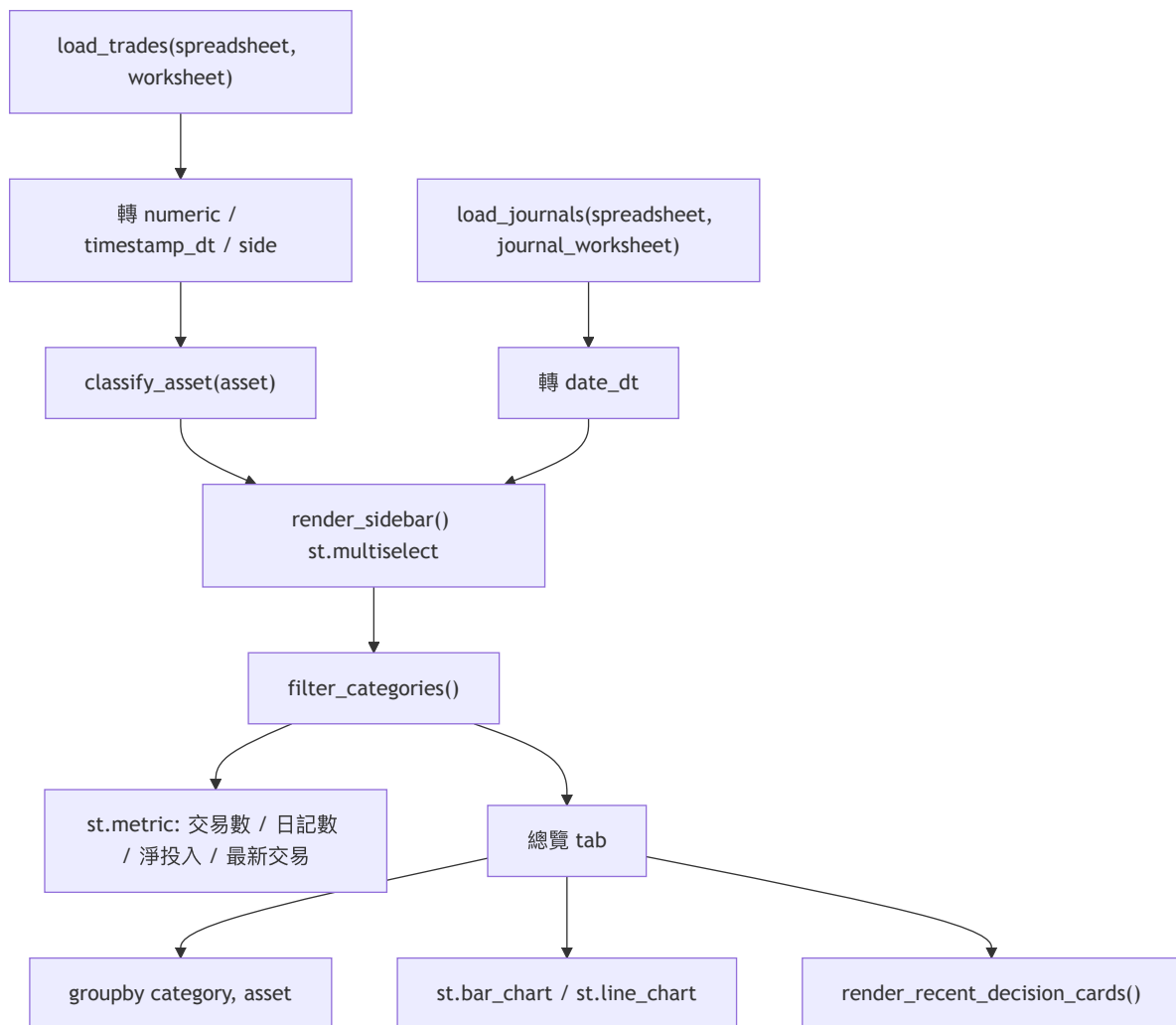
操作與業務流程

1. Streamlit 啟動與 Google Sheets 連線



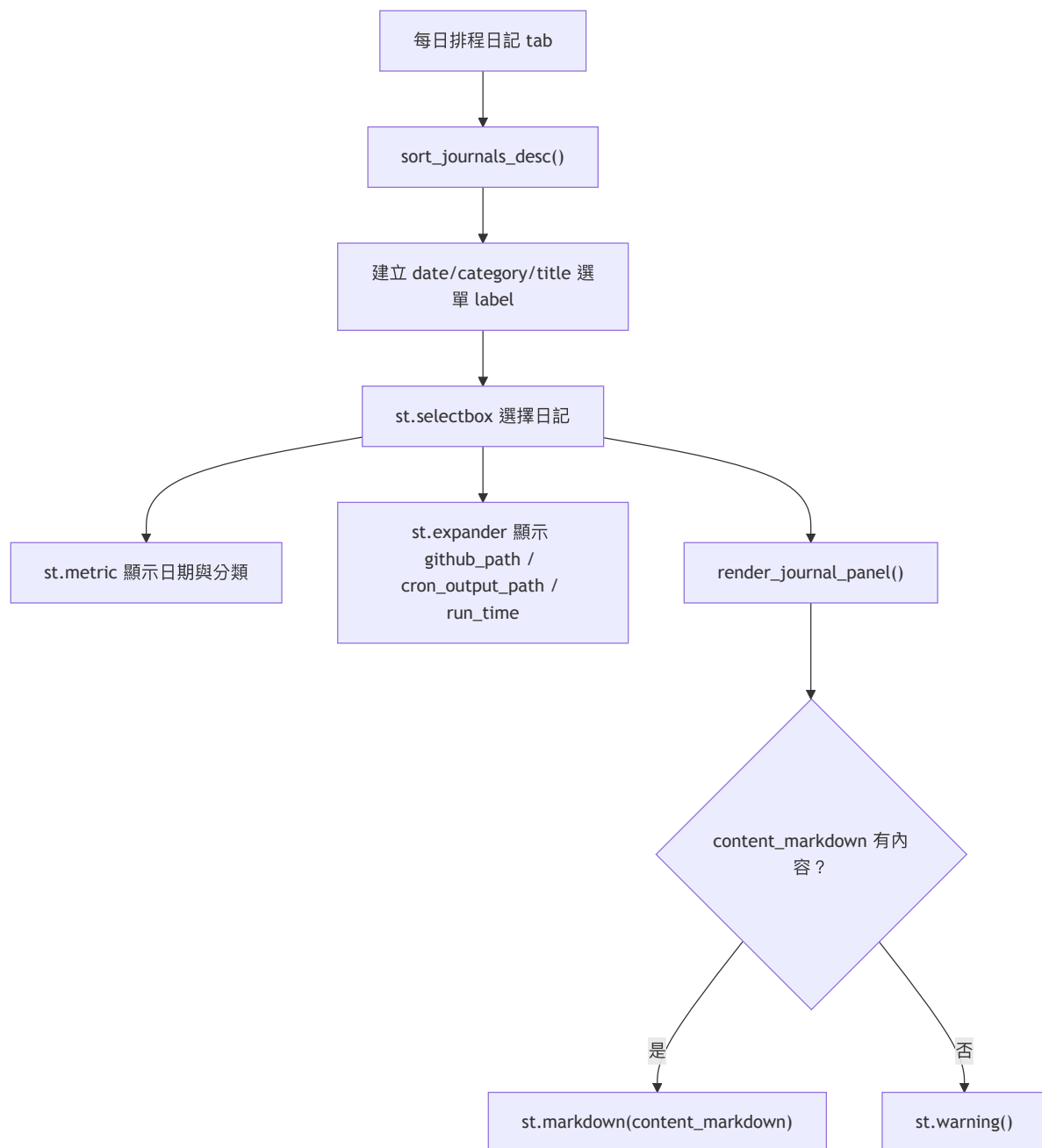
app.py 先檢查 Streamlit Secrets，再用 Google Service Account 建立 readonly 連線。缺少 secrets 時不載入資料，直接顯示設定說明並停止頁面流程。

2. Dashboard 分類篩選與總覽彙總



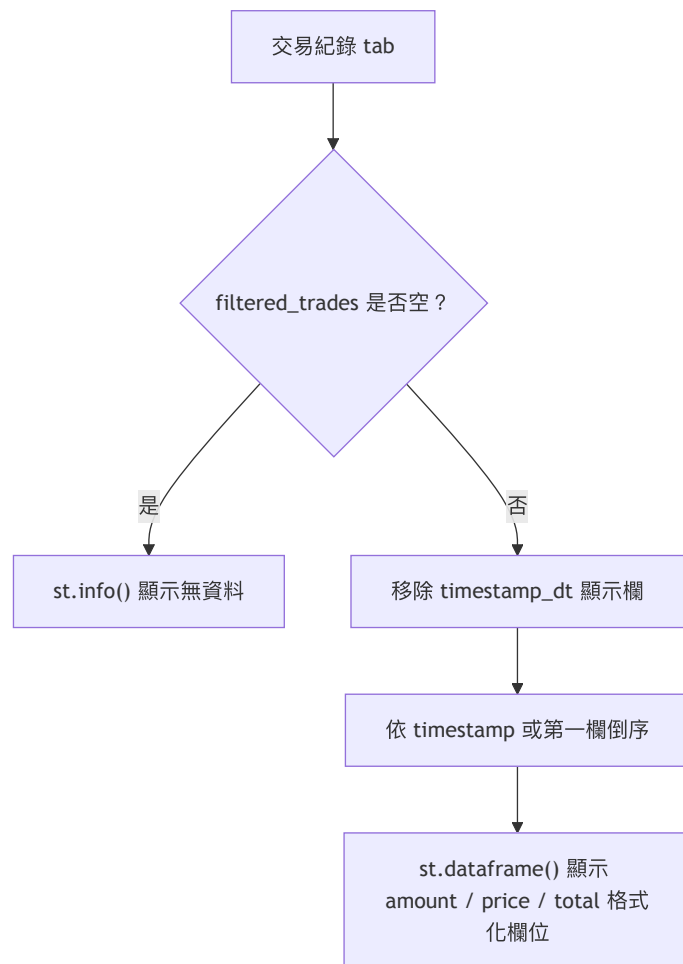
總覽依 sidebar 選取分類過濾交易與日記。交易會依資產代號分類並計算持有量、淨現金流、分類買賣金額與每日淨現金流。

3. 每日排程日記閱讀



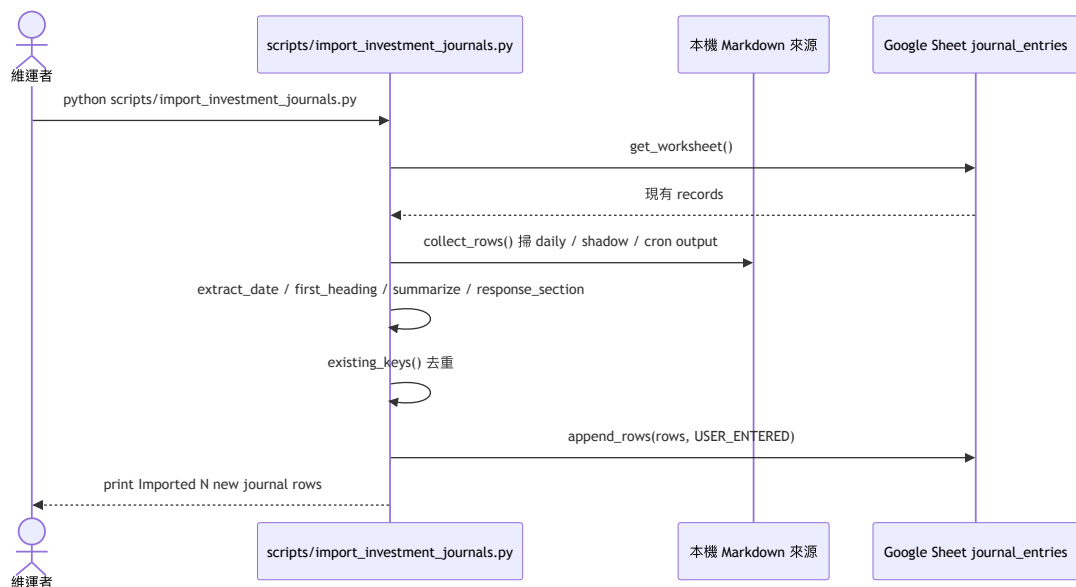
日記頁只讀取 `journal_entries`。使用者可從下拉選單切換日記，查看摘要、metadata 與完整 Markdown 內容。

4. 完整交易紀錄檢視



交易紀錄頁顯示 Google Sheet `trades` 的完整資料。程式只做格式化、排序與欄位顯示，不提供新增或更新交易的 UI。

5. 歷史投資日記匯入



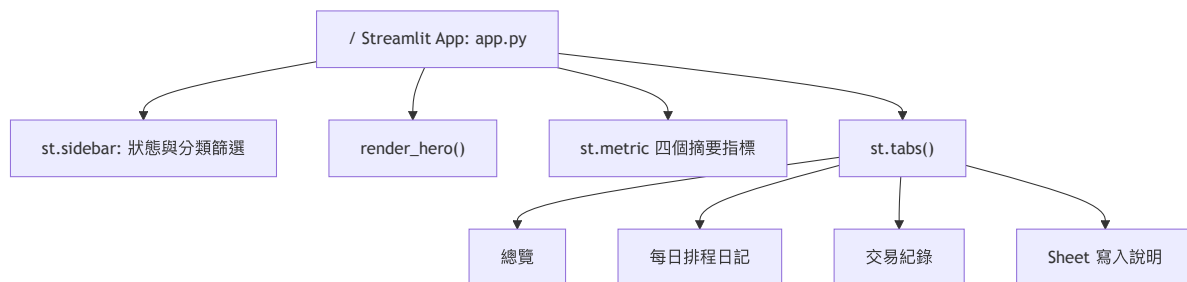
匯入腳本是本機維護工具，會用可寫入的 Google Sheets / Drive scopes。它只寫入 `journal_entries`，並用 `(date, source, github_path, cron_output_path)` 避免重複匯入。

「PAGES.md」

頁面、路由與模組清單

Streamlit 頁面結構

這個 repo 沒有 Flask、FastAPI、Next.js、Django 或自訂 HTTP routes。Web 介面是單一 `app.py` Streamlit app；頁面切換由 `st.tabs()` 完成，不是獨立 URL route。



路徑 / 頁面	程式位置	用途	是否需登入
/	app.py	Streamlit 單頁儀表板入口。載入 Google Sheet 後顯示 hero、摘要 metrics、sidebar 與 tabs。	否。程式碼沒有使用者登入判斷；資料連線由 Service Account secrets 控制。
總覽 tab	app.py with tab_overview:	顯示分類資產彙總、分類買賣金額、每日淨現金流、最近排程決策。	否
每日排程日記 tab	app.py with tab_journals:	用下拉選單閱讀 <code>journal_entries</code> 的日記、metadata 與 <code>content_markdown</code> 。	否
交易紀錄 tab	app.py with tab_trades:	顯示 <code>trades</code> worksheet 的完整交易資料。	否
Sheet 寫入說明 tab	app.py with tab_sheet:	說明網頁唯讀、手動修改需在 Google Sheet 操作、列出 <code>journal_entries</code> 欄位。	否

自訂 API Endpoint

方法	路徑	用途	狀態
無	無	repo 沒有實作對外 HTTP API endpoint。Streamlit runtime 會處理頁面服務，但不是本 repo 自訂 route。	不適用

外部 API 呼叫與輔助 Client

方法	目標	程式位置	用途	備註
Google Sheets API	<code>client.open_by_key()</code> , <code>spreadsheet.worksheet()</code> , <code>worksheet.get_all_records()</code>	app.py	唯讀載入 <code>trades</code> 與 <code>journal_entries</code> 。	使用 https://www.googleapis.com/auth/scope 。
Google Sheets API	<code>worksheet.append_rows()</code>	<code>scripts/import_investment_journals.py</code>	匯入 markdown 日記到 <code>journal_entries</code> 。	使用 <code>spreadsheets</code> 與 <code>drive</code> scopes
GET	<code>\${SMART_ROUTER_URL}/route</code>	<code>router_client.py</code> <code>_pick_model()</code>	依 tier 選擇模型。	主 app.py 未引用。
POST	<code>\${SMART_ROUTER_URL}/v1/chat/completions</code>	<code>router_client.py</code> <code>chat()</code>	將 messages 送到本地 AI Smart Router。	主 app.py 未引用； <code>httpx</code> 未列在 <code>req</code>

主要模組與函式

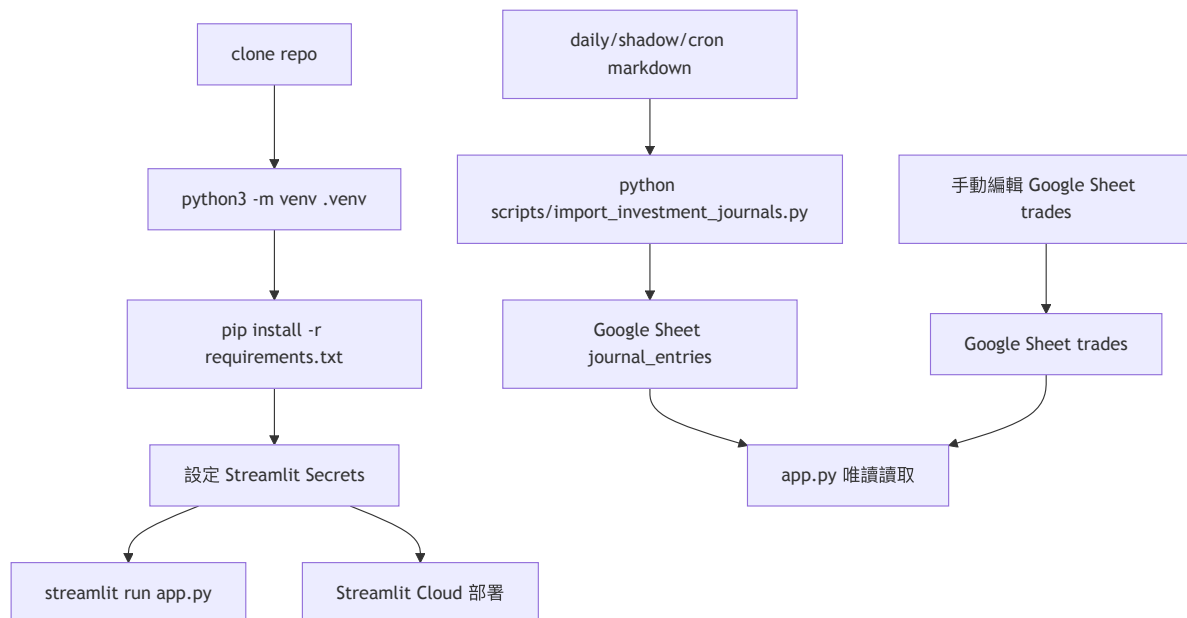
模組 / 函式	用途
<code>app.py: get_spreadsheet()</code>	讀取 Streamlit Secrets，建立 Google Sheets readonly 連線。
<code>app.py: _worksheet_records()</code>	讀取 worksheet records；worksheet 不存在或空資料時回傳空 DataFrame。
<code>app.py: load_trades()</code>	載入交易，轉換數字欄位、時間欄位、資產代號與分類。
<code>app.py: load_journals()</code>	載入日記，轉換日期欄位並補齊 header 欄位。
<code>app.py: classify_asset()</code>	將資產代號分為 <code>Crypto</code> 、 <code>ETF</code> 或 其他。
<code>app.py: filter_categories()</code>	依 sidebar 多選分類過濾交易與日記。

模組 / 函式	用途
<code>app.py:render_sidebar()</code>	顯示狀態、worksheet 名稱與分類篩選。
<code>app.py:render_hero()</code>	顯示儀表板主標題與資料狀態。
<code>app.py:render_recent_decision_cards()</code>	顯示最近排程決策卡片。
<code>scripts/import_investment_journals.py:collect_rows()</code>	掃描 daily、shadow、cron output markdown 並轉為 <code>JournalRow</code> 。
<code>scripts/import_investment_journals.py:existing_keys()</code>	讀取已存在紀錄並產生去重 key。
<code>scripts/import_investment_journals.py:main()</code>	執行匯入、去重與 append rows。
<code>router_client.py:chat()</code>	呼叫本地 AI Smart Router chat completions endpoint。

「OPERATIONS.md」

安裝、執行、部署、維運

維運總覽



環境需求

項目	需求
Python	repo 沒有版本鎖定檔；既有 README 使用 <code>python3</code> 。
套件管理	<code>pip</code> + <code>requirements.txt</code> 。
Google Cloud	需要 Service Account JSON 內容或本機憑證檔。
Google Sheet	需要至少 <code>trades</code> worksheet；日記功能使用 <code>journal_entries</code> worksheet。
Streamlit	本機用 <code>streamlit run app.py</code> ；部署用 Streamlit Cloud。

repo 未找到 `package.json`、`Makefile`、`Dockerfile`、`docker-compose.yml`、`Prisma schema` 或 `SQL migration`。

安裝與本機啟動

```
git clone https://github.com/wsx5031060310guy/crypto-diary.git
cd crypto-diary
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

本機執行前需建立 `.streamlit/secrets.toml`，內容格式同 Streamlit Cloud Secrets。不要把 `secrets` 檔 commit。

Streamlit Secrets

`app.py` 讀取 `st.secrets`，不是 `process.env` 或 `.env`。必要 secrets：

區塊 / key	用途	是否敏感
<code>[gcp_service_account]</code>	Google Service Account JSON 內容。	是
<code>type</code>	Service Account 類型。	否
<code>project_id</code>	Google Cloud project id。	視情況
<code>private_key_id</code>	private key id。	是
<code>private_key</code>	private key。	是

區塊 / key	用途	是否敏感
client_email	Service Account email，需要被加入 Google Sheet 權限。	視情況
client_id	Service Account client id。	視情況
auth_uri	Google OAuth auth URI。	否
token_uri	Google OAuth token URI。	否
auth_provider_x509_cert_url	Google cert URL。	否
client_x509_cert_url	Service Account cert URL。	視情況
universe_domain	Google API domain。	否
[sheet].id	Google Sheet ID。	視情況；不要公開到不受信任環境。
[sheet].worksheet	交易 worksheet 名稱；預設 trades。	否
[sheet].journal_worksheet	日記 worksheet 名稱；預設 journal_entries。	否

環境變數

repo 未找到 .env.example。程式碼讀取的環境變數如下；不要在文件或版本庫寫入真實憑證值。

變數	程式位置	用途	必要性
CRYPTO_BOT_SHEET_ID	scripts/import_investment_journals.py	匯入腳本要寫入的 Google Sheet ID。	匯入腳本建議設定；未設定時使用程式碼預設值。
CRYPTO_BOT_GCP_CRED	scripts/import_investment_journals.py	本機 Service Account JSON 檔路徑。	匯入腳本需要可讀的憑證檔。
CRYPTO_BOT_JOURNAL_WORKSHEET	scripts/import_investment_journals.py	匯入目標 worksheet；預設 journal_entries。	可選
AI_INVESTMENT_JOURNAL_DIR	scripts/import_investment_journals.py	daily/ 與 shadow/ markdown 日記來源根目錄。	可選
HERMES_INVESTMENT_CRON_OUTPUT	scripts/import_investment_journals.py	Hermes cron output markdown 來源目錄。	可選
SMART_ROUTER_URL	router_client.py	本地 AI Smart Router base URL；預設 http://127.0.0.1:8765。	只有使用 router_client.py 時需要

部署方式

既有 README 標示部署於 Streamlit Cloud：

- <https://crypto-diary-bvprx9psuxdvbuymohhgr.streamlit.app/>

部署需要：

1. Streamlit Cloud 指向此 repo。
2. App 入口使用 app.py。
3. 在 Streamlit Cloud App Settings → Secrets 設定 [gcp_service_account] 與 [sheet]。
4. Google Sheet 需把 Service Account email 加入讀取權限；若匯入腳本要寫入，該憑證需有寫入權限。

repo 沒有 Docker 或其他部署設定檔。

常見維運操作

匯入每日排程歷史紀錄

```
python scripts/import_investment_journals.py
```

在既有 README 記錄的 Mike Hermes 環境，可使用既有 crypto bot venv：

```
/Users/mike-hermes-ai/.hermes/crypto_bot/venv/bin/python \  
/Users/mike-hermes-ai/projects/crypto-diary/scripts/import_investment_journals.py
```

腳本會：

1. 讀取目標 Google Sheet 的 journal_entries。
2. 若 worksheet 不存在就建立並寫入 header。
3. 掃描 daily/*.md、shadow/*.md 與 Hermes cron output 20*.md。
4. 用 (date, source, github_path, cron_output_path) 去重。
5. 將新資料用 append_rows(..., value_input_option="USER_ENTERED") 寫入。

資料表維護

trades 欄位：

```
timestamp, asset, side, amount, price, total, note
```

journal_entries 欄位：

```
date, run_time, category, source, title, summary, content_markdown, github_path, cron_output_path, tags
```

app.py 不會建立或修改 worksheet。scripts/import_investment_journals.py 只會建立 / 補 header 給 journal_entries，且遇到既有 header 不符會 raise ValueError。

Migration、seed、cron、備份

操作	現況
Migration	無 migration 系統；schema 是 Google Sheet header。
Seed	無 seed 腳本。
Cron	repo 沒有 cron 設定檔；既有 README 描述外部 Hermes cron daily-investment-journal 每天 09:00 產生日記。
備份	repo 沒有備份腳本；可用 Google Sheets 版本記錄或手動匯出。
快取	load_trades() 與 load_journals() 使用 @st.cache_data(ttl=300)；資料變更後最多可能延遲 5 分鐘反映。